

Sistema de Contabilidade Pessoal por Partidas Dobradas

Programação C (COMP0512) — Turma T02 — 2026.2

Prof. Dr. André Yoshiaki Kashiwabara — andreyoshiaki@dcomp.ufs.br

Grupos	de 2 a 4 integrantes
Entregas	via GitHub (repositório privado, professor (ayoshiaki) como colaborador)
Apresentação final	14/12/2026, em sala
Peso na média	$MF = (2 \cdot P1 + 2 \cdot P2 + PJ)/5$

1 Visão geral

Ao longo do semestre, cada grupo desenvolverá em **linguagem C** um sistema de **contabilidade pessoal por partidas dobradas**: um programa de terminal capaz de registrar contas e lançamentos financeiros, manter saldos sempre consistentes, persistir os dados em disco e produzir relatórios que ajudem uma pessoa a entender sua vida financeira.

O projeto é dividido em **quatro fases**, com datas casadas com o conteúdo visto em aula. Cada fase amplia a anterior: vocês trabalharão sempre sobre o mesmo código, refatorando quando necessário — exatamente como acontece em software real.

Importante — sobre estruturas de dados. Este enunciado **não diz qual estrutura de dados usar em nenhuma fase**. Ele descreve *comportamentos e restrições* que o sistema deve satisfazer. Analisar os requisitos, escolher a estrutura adequada e **justificar a escolha por escrito** é parte central da avaliação. Uma solução que funciona, mas usa uma estrutura claramente inadequada aos requisitos da fase, perderá pontos; uma escolha bem argumentada, mesmo não sendo a “oficial”, será valorizada.

2 O domínio: partidas dobradas em 5 minutos

O método das partidas dobradas é usado há mais de 500 anos e é a base de toda a contabilidade moderna (e de programas como GnuCash, hledger e Beancount).

- O dinheiro é organizado em **contas**, de cinco tipos: **ativo** (o que você tem: carteira, conta corrente), **passivo** (o que você deve: cartão de crédito, financiamento), **receita** (o que entra: salário), **despesa** (o que sai: mercado, aluguel) e **patrimônio líquido** (saldos iniciais, ajustes).
- As contas formam uma **hierarquia** identificada por códigos, por exemplo:

```
1      Ativo
1.1    Ativo > Bancos
1.1.1  Ativo > Bancos > Banco do Brasil
1.1.2  Ativo > Bancos > Nubank
4      Despesa
```

4.1 Despesa > Moradia
 4.1.1 Despesa > Moradia > Aluguel

- Cada **lançamento** tem uma data, uma descrição e **duas ou mais partidas**. Cada partida credita ou debita um valor em uma conta. A regra de ouro: **em todo lançamento, a soma dos débitos é igual à soma dos créditos**. É isso que torna o sistema à prova de “dinheiro que some”.

Exemplo — pagar R\$ 120,00 de mercado com cartão de crédito:

Conta	Débito	Crédito
4.2.1 Despesa > Mercado	120,00	
2.1 Passivo > Cartão Visa		120,00

Exemplo — receber salário de R\$ 3.000,00 com desconto de R\$ 300,00 de INSS:

Conta	Débito	Crédito
1.1.1 Ativo > Banco do Brasil	2.700,00	
4.5 Despesa > INSS	300,00	
3.1 Receita > Salário		3.000,00

Exemplo — os **cinco tipos de conta** em um único lançamento. Primeiro dia de uso do sistema, que calha de ser dia de pagamento: você já tinha R\$ 1.500,00 na carteira e uma fatura de cartão de R\$ 1.200,00 em aberto (saldos iniciais, cuja contrapartida é o patrimônio líquido: o que você tinha menos o que devia, $1.500 - 1.200 = \text{R\$ } 300,00$); além disso, recebe salário de R\$ 3.000,00 com R\$ 300,00 de INSS descontado, com o líquido depositado no banco:

Conta	Débito	Crédito
1.1.1 Ativo > Banco do Brasil	2.700,00	
1.2 Ativo > Carteira	1.500,00	
4.5 Despesa > INSS	300,00	
2.1 Passivo > Cartão Visa		1.200,00
3.1 Receita > Salário		3.000,00
5.1 Patrimônio líquido > Saldos iniciais		300,00

Débitos: 4.500,00 = Créditos: 4.500,00 — a regra de ouro vale para qualquer número de partidas. (Na prática, saldos iniciais e salário seriam lançamentos separados; estão juntos aqui para mostrar os cinco tipos convivendo em um único lançamento balanceado.)

- O **saldo** de uma conta é o acumulado de seus débitos e créditos (para contas de ativo e despesa, débitos aumentam o saldo; para passivo, receita e patrimônio líquido, créditos aumentam). O saldo de uma conta intermediária (ex.: 1.1 Bancos) é a soma dos saldos de todas as suas subcontas.

Não é preciso saber contabilidade além do que está acima — o desafio do projeto é de **programação e estruturas de dados**, não de teoria contábil.

3 Regras gerais (valem para todas as fases)

1. **Linguagem e ferramentas:** C (padrão C11), compilado com `gcc -Wall -Wextra -std=c11` **sem warnings**. Apenas a biblioteca padrão do C é permitida. O projeto deve compilar e rodar em Linux com um único `make`.
2. **Precisão monetária:** somar milhares de valores não pode acumular erro de arredondamento — um balancete que fecha com diferença de R\$ 0,01 é um balancete errado. A representação interna de dinheiro é uma decisão de projeto do grupo e deve ser justificada no `DECISOES.md`.
3. **Memória:** toda memória alocada dinamicamente deve ser liberada. As entregas serão verificadas com `valgrind`; vazamentos e acessos inválidos descontam pontos.
4. **Robustez:** entrada inválida do usuário (data malformada, conta inexistente, valor negativo, lançamento que não balanceia) nunca pode derrubar o programa nem corromper os dados já registrados.
5. **DECISOES.md:** arquivo na raiz do repositório, atualizado **a cada fase**, com uma seção por fase contendo: (a) quais estruturas de dados foram escolhidas e para quê; (b) por que elas atendem aos requisitos da fase melhor que as alternativas consideradas; (c) qual o custo (tempo de execução) das operações principais sobre essas estruturas. Não precisa de formalismo — precisa de argumento.
6. **Autoria:** todo integrante deve ser capaz de explicar **qualquer** trecho do código entregue. Haverá arguição individual na apresentação final, e a nota individual pode ser ajustada em função dela e do histórico de commits.

4 Entrega via GitHub

- **Até 02/09/2026:** cada grupo cria um **repositório privado** no GitHub, adiciona o professor como colaborador e envia o link pelo SIGAA, junto com a lista de integrantes (que também deve constar no `README.md`).
- Cada fase é entregue criando uma **tag de release** no repositório (`fase-1`, `fase-2`, `fase-3`, `final`) até as 23h59 da data da entrega. O que estiver na tag é o que será avaliado.
- O `README.md` deve manter instruções atualizadas de compilação e uso.
- **O histórico de commits é parte da entrega.** Espera-se desenvolvimento contínuo, com commits significativos de **todos** os integrantes ao longo da fase — não um único commit gigante na véspera. Recomenda-se o uso de branches e pull requests entre os membros, mas não é obrigatório.
- **Atraso:** até 3 dias corridos, com desconto de 1,0 ponto (da fase) por dia. Após 3 dias, a fase recebe nota zero (o conteúdo continua sendo necessário para as fases seguintes).

5 As fases

5.1 Fase 1 — Fundações: contas, lançamentos e saldos

Entrega: 07/10/2026 (tag fase-1)

Conteúdo de apoio: registros e enumerações, ponteiros, alocação dinâmica de memória, modularização.

O programa, operado por um menu de terminal, deve permitir:

1. **Plano de contas:** criar contas informando código hierárquico (ex.: 1.1.2), nome e tipo (um dos cinco tipos da Seção 2); listar o plano de contas de forma legível, evidenciando a hierarquia; impedir códigos duplicados e rejeitar contas cujo código pai não exista.
2. **Lançamentos:** registrar lançamentos com data, descrição e partidas (quantas o usuário quiser, no mínimo duas), validando a regra de ouro antes de aceitar; listar todos os lançamentos registrados.
3. **Saldos:** exibir o saldo de qualquer conta informada.

Restrições que devem guiar as escolhas de vocês:

- O número de contas e de lançamentos é **desconhecido a priori** e não pode ter limite fixo: o sistema deve suportar 10 ou 10.000 lançamentos sem recompilar e sem desperdiçar memória de forma grosseira.
- Um lançamento não sabe de antemão quantas partidas terá.
- O código deve estar **modularizado** em múltiplos arquivos `.c/.h` coerentes (por exemplo: contas, lançamentos, interface — a divisão exata é decisão do grupo), com include guards e um `Makefile` funcional.

5.2 Fase 2 — O razão vivo: histórico, correções e hierarquia

Entrega: 18/11/2026 (tag fase-2)

Conteúdo de apoio: recursividade, estruturas de dados fundamentais, tipos abstratos de dados.

A vida financeira real não acontece em ordem: nota fiscal esquecida no bolso, extrato conferido semanas depois, lançamento digitado errado. Nesta fase o sistema passa a conviver com isso:

1. **Ordem cronológica com lançamentos retroativos:** os lançamentos devem ser mantidos **sempre em ordem de data**. Inserir um lançamento antigo (ex.: de três meses atrás) no meio de um histórico com milhares de registros **não pode exigir deslocar ou copiar os lançamentos já existentes**.
2. **Extrato navegável:** o usuário deve poder percorrer o extrato de uma conta lançamento a lançamento, **avanchando e retrocedendo** no tempo a partir de qualquer ponto, sem que retroceder exija percorrer tudo desde o início.
3. **Desfazer/refazer:** as operações de inclusão, edição e remoção de lançamentos devem poder ser **desfeitas em ordem inversa** (a última operação é a primeira a ser desfeita), e refeitas se o usuário mudar de ideia.
4. **Lançamentos agendados:** o usuário pode agendar lançamentos futuros (ex.: aluguel do mês que vem). Ao avançar a “data atual” do sistema, os agendamentos vencidos são efetivados **na ordem em que vencem** — o primeiro a vencer é o primeiro a ser efetivado.
5. **Saldos hierárquicos com recursividade:** o saldo de uma conta deve agregar os saldos de todas as suas subcontas, em qualquer profundidade (1 soma 1.1, que soma 1.1.1, 1.1.2, ...). A travessia da hierarquia deve ser implementada de forma **recursiva**. Um relatório de saldos indentado por nível deve exibir a árvore de contas completa.

6. **Tipos abstratos de dados:** as coleções centrais do sistema devem ser reescritas como **TADs opacos**: o `.h` expõe apenas o tipo e as operações; a `struct` fica definida no `.c`. Nenhum módulo cliente pode acessar campos internos das coleções.

No `DECISOES.md`, além das justificativas, inclua uma tabela com o custo das operações dos itens 1–4 na estrutura escolhida por vocês.

5.3 Fase 3 — Dados que sobrevivem: arquivos e flags

Entrega: 07/12/2026 (tag fase-3)

Conteúdo de apoio: operadores bit-a-bit, arquivos texto e binários.

1. **Persistência:** o estado completo do sistema (plano de contas, lançamentos, agendamentos) deve ser salvo em um **arquivo binário** com formato definido pelo grupo — documentado no `DECISOES.md` — contendo número mágico e versão do formato. Ao abrir o programa, os dados são recarregados exatamente como estavam. Um arquivo truncado ou corrompido deve ser detectado e rejeitado com mensagem clara, sem travar o programa.
2. **Importação de extrato:** o programa deve importar um **arquivo texto CSV** no formato `data;descricao;valor` (extrato bancário simplificado), pedindo ao usuário a conta de contrapartida de cada linha — ou de todas — e gerando os lançamentos correspondentes. Linhas malformadas são reportadas e puladas.
3. **Atributos de lançamento com bits:** cada lançamento passa a ter atributos independentes e combináveis: *conciliado*, *estornado*, *agendado*, *importado*, *marcado para revisão*. Todos devem ser armazenados **em um único campo inteiro**, manipulados exclusivamente com operadores bit-a-bit (máscaras, `|`, `&`, `~`, `<<`), inclusive dentro do arquivo binário. O usuário deve poder marcar/desmarcar atributos e filtrar listagens por combinações deles (ex.: “importados e não conciliados”).

5.4 Fase 4 — Relatórios e apresentação final

Entrega: tag final até 13/12/2026 — Apresentação: 14/12/2026

Conteúdo de apoio: todo o semestre.

O produto final da contabilidade são os relatórios. O sistema deve oferecer, **no mínimo, quatro relatórios**, sendo dois obrigatórios e **pelo menos dois criados pelo grupo**.

Obrigatórios:

1. **Balancete de verificação:** todas as contas com seus saldos devedores e credores em um período; os totais devem fechar (é a prova pública de que as partidas dobradas funcionam).
2. **Extrato de conta por período:** lançamentos de uma conta entre duas datas, com saldo acumulado linha a linha.

Livres (escolham pelo menos dois — criatividade conta ponto):

- Fluxo de caixa mês a mês; despesas por categoria com gráfico de barras em ASCII; comparativo orçado × realizado; evolução do patrimônio líquido; “maiores vilões” do orçamento; resumo anual estilo “retrospectiva”; exportação de qualquer relatório para CSV; ou qualquer outra ideia que os dados de vocês permitam contar uma boa história.

Apresentação (14/12). 10 a 15 minutos por grupo, com demonstração ao vivo do sistema rodando sobre uma massa de dados realista (mínimo de 3 meses de movimentação simulada), apresentação dos relatórios e defesa das decisões de projeto (estruturas de dados escolhidas em cada fase e por quê). Em seguida, arguição individual: qualquer integrante pode ser perguntado sobre qualquer parte do código.

6 Avaliação

A nota do projeto (PJ, de 0 a 10) é composta por:

Componente	Peso
Fase 1	20%
Fase 2	30%
Fase 3	25%
Fase 4 (relatórios + apresentação + defesa)	25%

Dentro de cada fase:

Critério	Peso
Funcionalidade correta (requisitos da fase)	50%
Adequação e justificativa das estruturas de dados (<code>DECISOES.md</code>)	20%
Qualidade do código: modularização, padrões, legibilidade	15%
Corretude de memória (<code>valgrind</code>) e robustez	10%
Uso do GitHub: commits contínuos e distribuídos, README, tags	5%

A nota é do grupo, mas pode ser **individualizada**: integrante sem participação verificável (commits + arguição) pode receber nota inferior à do grupo, inclusive zero.

7 Perguntas frequentes

Podemos usar bibliotecas prontas de listas/estruturas? Não. Implementar as estruturas é o objetivo da disciplina. Apenas a biblioteca padrão do C.

Podemos mudar uma estrutura escolhida em fase anterior? Sim — refatorar faz parte. Registre a mudança e o motivo no `DECISOES.md`.

Podemos usar estruturas não vista em aula? Sim, o grupo poderá pesquisar e implementar estruturas de dados diferentes daquelas visto em aula. Registre em `DECISOES.md`

E se o grupo se desfizer no meio do semestre? Procurem o professor o quanto antes; a divisão será tratada caso a caso, preservando o histórico de commits de cada um.

Podemos usar ferramentas de IA? Como apoio ao estudo, sim. Mas a regra do item 6 da Seção 3 é absoluta: na arguição, cada integrante responde pelo código como seu. Não saber explicar o próprio código equivale a não tê-lo feito.